

Lecture 14: Training Transformers

Week 5, Lecture 3 - From Architecture to Implementation

PSYC 51.07: Models of Language and Communication

Winter 2026

Today's Agenda

1.  **Positional Encoding:** Injecting sequence order
2.  **Feed-Forward Networks:** The other key component
3.  **Layer Norm & Residuals:** Training deep networks
4.  **FlashAttention:** Making transformers faster
5.  **Three Architectures:** Encoder, Decoder, Both
6.  **Practical Implementation:** Using HuggingFace

Goal: Complete understanding of transformer training and implementation

Positional Encoding

Problem: Self-attention is permutation-invariant!

Without position information:

- "The cat sat" = "sat cat the" = "cat the sat"
- Order matters in language!

Solution: Add positional encodings to input embeddings

Original Transformer (Sinusoidal):

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

Properties:

- Deterministic
- Unique for each position
- Smooth changes
- Generalizes to unseen lengths

Modern Approach: RoPE

Rotary Position Embedding (Su et al. 2021)

- Rotates Q and K in complex space
- Relative position encoding
- Better extrapolation
- Used in: LLaMA, GPT-NeoX

Learned Positional Embeddings:

- Treat positions as vocabulary
- Learn embeddings during training
- Used in: BERT, GPT

References: Vaswani et al. (2017), Su et al. (2021) - "RoFormer: Enhanced Transformer with Rotary Position Embedding"

Why Sinusoidal Positional Encoding?

Advantages of sine/cosine functions:

1. Unique Patterns

- Each position gets a unique vector
- Different frequencies for different dimensions
- Creates distinguishable position signatures

2. Relative Position Information

- PE_{pos+k} can be expressed as linear function of PE_{pos}
- Model can learn to attend by relative position
- Trigonometric identity: $\sin(a + b) = \sin a \cos b + \cos a \sin b$

3. Extrapolation

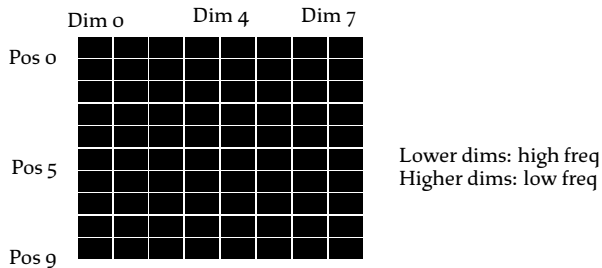
- Can handle sequences longer than training length
- Sine/cosine continue smoothly
- No need to retrain for different lengths

4. No Additional Parameters

- Deterministic, no learning required
- Saves memory compared to learned embeddings

Visualizing Positional Encodings

Each position gets a unique pattern across dimensions



Key Insight:

- Lower dimensions: Rapid oscillation (high frequency)
- Higher dimensions: Slow oscillation (low frequency)
- Creates a unique "barcode" for each position
- Model learns to use these patterns for positional awareness

Purpose: Add non-linearity and transform representations

After attention, apply position-wise feed-forward network

Architecture:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- Two linear transformations with ReLU activation
- Applied to each position independently
- Same network for all positions
- Typical: Expand 4x then project back

Why Feed-Forward Networks? 🤔

Role in the Transformer:

1. Add Non-linearity

- Attention is mostly linear operations (weighted sums)
- FFN introduces non-linear transformations
- ReLU/GELU activation adds expressiveness

2. Position-wise Processing

- Attention mixes information across positions
- FFN processes each position independently
- Allows position-specific feature transformations

3. Increase Model Capacity

- Expansion (4x) provides more parameters
- Can learn complex feature combinations
- Most parameters in transformer are in FFN layers!

4. Feature Refinement

- Attention gathers context
- FFN refines and transforms the representation
- Two complementary operations

Layer Normalization & Residual Connections

Critical for training deep transformers!

Residual Connections:

$$\text{Output} = \text{Layer}(x) + x$$

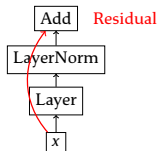
- Allows gradients to flow directly
- Prevents vanishing gradients
- Enables very deep networks
- Identity mapping as fallback

Analogy: Like highway roads for information flow!

Layer Normalization:

$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sigma} + \beta$$

- Normalize across features
- Stabilizes training
- Faster convergence
- Independent of batch size



Standard Pattern:

$$x \leftarrow \text{LayerNorm}(x + \text{MultiHeadAttention}(x))$$

$$x \leftarrow \text{LayerNorm}(x + \text{FFN}(x))$$

Pre-Norm vs Post-Norm

Two ways to arrange LayerNorm and residual connections

Post-Norm (Original):

$$x \leftarrow \text{LN}(x + \text{Attn}(x))$$

$$x \leftarrow \text{LN}(x + \text{FFN}(x))$$

- Used in original Transformer
- Normalization after residual
- Better performance when it works
- **Can be unstable for deep models**
- Requires careful initialization

Pre-Norm (Modern):

$$x \leftarrow x + \text{Attn}(\text{LN}(x))$$

$$x \leftarrow x + \text{FFN}(\text{LN}(x))$$

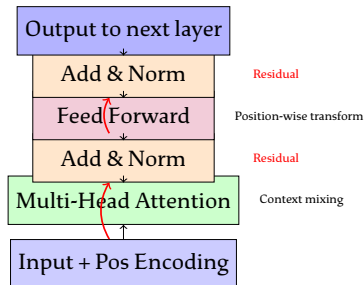
- Increasingly popular
- Normalization before sublayer
- **More stable training**
- Easier to train very deep models
- Used in: GPT-2, GPT-3, many modern LLMs

Recommendation

For deep transformers (24+ layers), Pre-Norm is generally preferred due to better training stability.

Complete Transformer Block

Putting it all together:



Stack this block N times!

- BERT-base: 12 blocks
- BERT-large: 24 blocks
- GPT-3: 96 blocks!

FlashAttention: Making Transformers Faster ⚡

Problem: Standard attention is slow and memory-hungry!

Standard Attention Complexity

- Time: $O(n^2)$ where n is sequence length
- Memory: $O(n^2)$ to store attention matrix
- Bottleneck: Reading/writing to GPU memory (HBM)

FlashAttention Innovation:

- Tile-based computation
- Uses fast SRAM instead of slow HBM
- Fused operations
- Recomputation in backward pass
- **2-4x faster!**
- Enables longer sequences

Impact:

- GPT-3: 512 → 2048 context
- BERT: Faster training
- Long-document understanding
- Used in: LLaMA, GPT-4

Method	Speed	Memory
Standard	1x	1x
FlashAttn	3x	0.5x

Reference: Dao et al. (2022) - "FlashAttention: Fast and Memory-Efficient Exact Attention"

Other Attention Optimizations

Addressing the $O(n^2)$ problem:

1. Sparse Attention

- Only attend to subset of positions
- Local windows + global tokens
- Used in: Longformer, BigBird

2. Linear Attention

- Approximate attention with linear complexity
- Kernel trick to avoid materializing attention matrix
- Used in: Performer, Linear Transformer

3. Low-Rank Approximation

- Factorize attention matrix
- Reduce memory footprint
- Used in: Linformer

4. Sliding Window

- Fixed-size local attention window
- Constant memory usage
- Used in: Mistral 7B

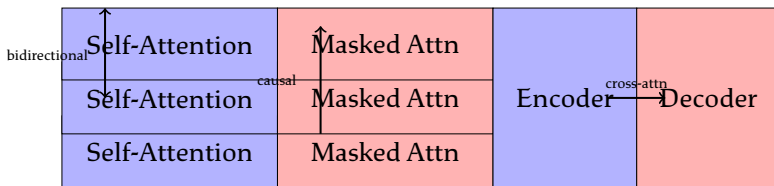
Trade-off: Efficiency vs. expressiveness. Full attention often still best for quality.

Three Transformer Architectures

Architecture	How it works	Examples & Uses
Encoder-only <i>Best for:</i> Classification, NER, QA	Bidirectional self-attention. Can see entire sequence.	BERT, RoBERTa, AL-BERT
Decoder-only <i>Best for:</i> Generation, completion	Masked self-attention. Can only see past. Autoregressive.	GPT, GPT-2, GPT-3, LLaMA
Encoder-Decoder Decoder: masked + cross-attention	Encoder: bidirectional T5, BART, mT5	

Visual Comparison: Encoder vs Decoder vs Both

Encoder-Only (BERT) Decoder-Only (GPT) Encoder-Decoder (T5)



Choosing the Right Architecture:

- Need to understand full context? → **Encoder** (BERT)
- Need to generate text? → **Decoder** (GPT)
- Need both (translate, summarize)? → **Encoder-Decoder** (T5)

HuggingFace makes it easy!

```
from transformers import AutoModel, AutoTokenizer

# Load pre-trained model
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Tokenize input
text = "The animal didn't cross the street because it was too
        tired"
inputs = tokenizer(text, return_tensors="pt")

# Get contextualized embeddings
outputs = model(**inputs)
hidden_states = outputs.last_hidden_state # Shape: [batch,
```

Different architectures for different tasks

```
from transformers import AutoModelForSequenceClassification, \
                        AutoModelForCausalLM, \
                        AutoModelForSeq2SeqLM

# 1. ENCODER-ONLY (BERT): Classification
encoder_model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2
)
# Use for: Sentiment analysis, NER, classification

# 2. DECODER-ONLY (GPT): Text generation
decoder_model = AutoModelForCausalLM.from_pretrained("gpt2")
# Use for: Text completion, creative writing, few-shot learning
```

Practical Tips for Training Transformers

1. Learning Rate & Warmup

- Use learning rate warmup (linear increase then decay)
- Typical: warmup for 10% of training steps
- Peak LR: $1e-4$ to $5e-4$ (from scratch), $1e-5$ to $5e-5$ (fine-tuning)

2. Optimization

- Use Adam or AdamW optimizer
- AdamW: Adam + weight decay (better generalization)
- Gradient clipping to prevent exploding gradients

3. Regularization

- Dropout after attention and FFN (typical: 0.1)
- Weight decay (typical: 0.01)
- Label smoothing for classification

4. Mixed Precision Training

- Use FP16 instead of FP32
- 2x speedup, 2x memory reduction
- Minimal accuracy loss

1. Batch Size

- Larger batches = better GPU utilization
- Use gradient accumulation if GPU memory limited
- Typical: effective batch size 256-2048 tokens

2. Sequence Length

- Shorter sequences train faster (quadratic complexity!)
- Consider truncation or sliding windows
- Pack multiple examples to maximize GPU usage

3. Model Size

- Start small, scale up if needed
- DistilBERT: 40% smaller, 60% faster, 97% performance
- Consider model distillation for deployment

4. Hardware

- GPUs with high memory bandwidth (A100, H100)
- Multi-GPU training with data parallelism
- Use FlashAttention when available

Discussion Questions

1. Positional Encoding:

- Why add instead of concatenate?
- What happens without positional encoding?
- Learned vs. fixed: which is better?

2. Architecture Choice:

- When would you use encoder-only vs decoder-only?
- Can GPT do classification? Can BERT generate?
- Why has decoder-only become more popular recently?

3. Scaling:

- Is bigger always better?
- What are the limits to scaling transformers?
- How do we make them more efficient?

4. Training Stability:

- Why are residual connections so important?
- Pre-norm vs post-norm: trade-offs?

Looking Ahead to Week 6

This week (Week 5) we learned:

- Attention mechanisms (Lecture 12)
- Self-attention and transformer architecture (Lecture 13)
- Training transformers: all the components (Lecture 14)

Next week (Week 6):

- **Lecture 15 - BERT Deep Dive**
 - Masked Language Modeling
 - Pre-training and fine-tuning
 - Contextual embeddings in action
- **Lecture 16 - BERT Variants**
 - RoBERTa, ALBERT, DistilBERT
 - Improvements and optimizations
- **Lecture 17 - Applications of Encoder Models**
 - Real-world BERT applications
 - Cognitive neuroscience connections
 - Understanding vs. pattern matching

Assignment 4: Building context-aware systems (check syllabus for details)

Key Takeaways:

1. Positional Encoding

- Sine/cosine functions inject position information
- Enables model to understand order

2. Feed-Forward Networks

- Position-wise transformations
- Add non-linearity and capacity

3. LayerNorm & Residuals

- Critical for training deep networks
- Stabilize gradients, enable deeper models

4. Three Architectures

- Encoder (BERT), Decoder (GPT), Both (T5)
- Choose based on task requirements

5. Practical Considerations

- Use HuggingFace for easy implementation
- FlashAttention for efficiency
- Careful hyperparameter tuning

Essential Papers:

- **Vaswani et al. (2017)** - "Attention Is All You Need"
 - The original Transformer paper
- **Su et al. (2021)** - "RoFormer: Enhanced Transformer with Rotary Position Embedding"
 - Modern positional encoding approach
- **Dao et al. (2022)** - "FlashAttention: Fast and Memory-Efficient Exact Attention"
 - Making transformers faster

Tutorials:

- HuggingFace Course: Chapters 1.4, 1.5
- The Illustrated Transformer:
<https://jalammar.github.io/illustrated-transformer/>
- Annotated Transformer: <https://nlp.seas.harvard.edu/annotated-transformer/>

Discussion Time

Topics for discussion:

- Positional encoding approaches
- Architecture choices
- Training tips and tricks
- Implementation questions
- Assignment 4 preparation

Thank you! 

See you next week for BERT!